Routage et Navigation dans une SPA avec React Router

Objectifs

1. Comprendre le fonctionnement du routage côté client (*Client-Side Routing*) via l'API History du navigateur.
2. Installer et configurer `react-router-dom` au sein d'une Single Page Application.
3. Déclarer une arborescence de routes statiques et imbriquées (*Layouts* partagés avec `<Outlet />`).
4. Naviguer de manière déclarative (`<Link>`, `<NavLink>`) et impérative (`useNavigate`) sans recharger la page.
5. Extraire et exploiter des paramètres d'URL dynamiques avec `useParams` et gérer les pages introuvables (`404`) ainsi que les redirections (`<Navigate>`).

1. Le Défi de la Navigation dans une Single Page Application

1.1 Routage Serveur Classique vs Routage Client (SPA)

Dans une application web traditionnelle (Multi-Page Application), chaque URL (`/accueil`, `/stations`, `/alertes`) correspond à une requête HTTP envoyée au serveur, qui renvoie un nouveau document HTML complet.

Dans une **Single Page Application (SPA)** React :

- Le serveur ne renvoie qu'un seul fichier `index.html`.
- Si l'utilisateur utilise une balise HTML standard `<a href="/alertes">`, le navigateur détruit toute la page actuelle, recharge intégralement le bundle JavaScript et **réinitialise tout votre Store Redux et vos états locaux à zéro**.

Pour éviter cela, on utilise le **Routage Côté Client (*Client-Side Routing*)** via la bibliothèque standard **`react-router-dom`** :

1. Elle intercepte les changements d'URL dans la barre d'adresse grâce à l'API `window.history` (HTML5 History API).
2. Elle empêche la requête réseau vers le serveur.

3. Elle monte ou démonte dynamiquement les composants React correspondant à l'URL active en quelques millisecondes, tout en préservant l'état global en mémoire.

Installation

```shell
npm install react-router-dom
```

2. Mise en Place de Base : `<BrowserRouter>`, `<Routes>` et `<Route>`

2.1 Envelopper l'Application avec `<BrowserRouter>`

Tout comme Redux nécessite un `<Provider>`, React Router a besoin d'envelopper l'application à sa racine (dans `src/main.jsx`) avec le composant `<BrowserRouter>` pour synchroniser l'interface avec l'URL du navigateur.

```javascript
// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux';
import { BrowserRouter } from 'react-router-dom';
import { store } from './store/store.js';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </Provider>
  </StrictMode>
);
```

2.2 Déclarer la Table de Routage

Dans votre composant principal `App.jsx`, vous utilisez le conteneur `<Routes>` qui évalue l'URL actuelle et affiche l'unique `<Route>` dont l'attribut `path` correspond le mieux :

JavaScript

```
// src/App.jsx
import { Routes, Route } from 'react-router-dom';
import DashboardPage from './pages/DashboardPage';
import AlertsPage from './pages/AlertsPage';
import NotFoundPage from './pages/NotFoundPage';

export default function App() {
  return (
    <div className="app-shell">
      <Routes>
        {/* Route racine */}
        <Route path="/" element={<DashboardPage />} />

        {/* Route dédiée aux alertes */}
        <Route path="/alerts" element={<AlertsPage />} />

        {/* Route générique "Catch-all" pour les erreurs 404 */}
        <Route path="*" element={<NotFoundPage />} />
      </Routes>
    </div>
  );
}
```

3. Navigation Déclarative : `<Link>` et `<NavLink>`

3.1 Le Composant `<Link>`

Pour créer des liens de navigation internes sans provoquer de rechargement de page, **n'utilisez jamais `<a href="...">`**. Remplacez-le systématiquement par le composant `<Link to="...">`.

JavaScript

```
import { Link } from 'react-router-dom';

export function QuickNav() {
  return (
    <nav>
      <Link to="/">Tableau de bord</Link>
      <Link to="/alerts">Centre d'alertes</Link>
    </nav>
  );
}
```

3.2 Le Composant `<NavLink>` pour les Menus Actifs

`<NavLink>` est une version enrichie de `<Link>` conçue spécifiquement pour les barres de navigation. Il sait automatiquement si la route vers laquelle il pointe est actuellement active et permet de lui appliquer une classe CSS ou un style spécifique via une fonction `{ isActive }`.

JavaScript
```
import { NavLink } from 'react-router-dom';

export default function Navbar() {
  return (
    <nav className="main-nav">
      <NavLink
        to="/"
        className={({ isActive }) => (isActive ? 'nav-link active' : 'nav-link')}
      >
        Supervision
      </NavLink>

      <NavLink
        to="/alerts"
        className={({ isActive }) => (isActive ? 'nav-link active' : 'nav-link')}
      >
        Alertes Réseau
      </NavLink>
    </nav>
  );
}
```

4. Routes Dynamiques et Lecture des Paramètres (`useParams`)

Dans une application métier, vous ne créez pas une route fixe par station météo (`/stations/1`, `/stations/2`, `/stations/3`). Vous définissez un **segment dynamique** préfixé par deux-points (`:stationId`).

4.1 Déclaration du Segment Dynamique

JavaScript
```
<Route path="/stations/:stationId" element={<StationDetailPage />} />
```

4.2 Extraction du Paramètre avec le Hook `useParams`

Dans le composant cible (`StationDetailPage`), le Hook **`useParams`** retourne un objet contenant les paramètres extraits de l'URL actuelle.

Note importante : Les valeurs issues de l'URL sont toujours des chaînes de caractères (`string`). Si vos identifiants sont numériques dans Redux, pensez à les convertir avec `Number(stationId)`.

JavaScript

```
// src/pages/StationDetailPage.jsx
import { useParams, Link } from 'react-router-dom';
import { useSelector } from 'react-redux';

export default function StationDetailPage() {
  // 1. Récupération du paramètre dynamique depuis l'URL
  const { stationId } = useParams();

  // 2. Recherche de la station correspondante dans le Store Redux
  const station = useSelector(state =>
    state.stations.list.find(st => st.id === Number(stationId))
  );

  if (!station) {
    return (
      <div className="card">
        <h2>Station introuvable (ID : {stationId})</h2>
        <Link to="/">Retour au tableau de bord</Link>
      </div>
    );
  }

  return (
    <article className="card">
      <h2>Fiche Technique : {station.name}</h2>
      <p>Identifiant : #{station.id}</p>
      <p>Altitude : {station.altitude} m</p>
      <p>Statut : {station.status}</p>
      <Link to="/">← Retour à la liste</Link>
    </article>
  );
}
```

5. Routes Imbriquées, Layouts Partagés (`<Outlet />`) et Redirections

5.1 Créer un Gabarit Commun avec `<Outlet />`

La plupart des applications partagent une structure commune (en-tête, barre latérale, pied de page) sur toutes les vues. Plutôt que de répéter `<Header />` dans chaque page, React Router permet d'imbriquer des routes à l'intérieur d'une route parente servant de **Layout** (gabarit).

Le composant **`<Outlet />`** indique l'emplacement exact où le composant de la route enfant active doit être injecté.

JavaScript

```
// src/components/MainLayout.jsx
import { Outlet } from 'react-router-dom';
import Header from './Header';
import Navbar from './Navbar';

export default function MainLayout() {
  return (
    <div className="container">
      <Header />
      <Navbar />
      <main className="page-content">
        {/* Les pages enfants s'afficheront exactement ici */}
        <Outlet />
      </main>
    </div>
  );
}
```

Configuration des routes imbriquées dans `App.jsx` :

JavaScript

```
<Routes>
  <Route path="/" element={<MainLayout />}>
    {/* Route par défaut ("index") lorsque l'URL est exactement "/" */}
    <Route index element={<DashboardPage />} />
    <Route path="stations/:stationId" element={<StationDetailPage />} />
    <Route path="alerts" element={<AlertsPage />} />
    <Route path="*" element={<NotFoundPage />} />
  </Route>
```

```
</Routes>
```

5.2 Navigation Impérative (`useNavigate`) et Redirection (`<Navigate>`)

Il existe deux façons de rediriger automatiquement l'utilisateur :

1. **Suite à une action ou un événement (`useNavigate`) :** Par exemple, rediriger l'opérateur vers l'accueil après avoir supprimé une station ou validé un formulaire.

```
JavaScript
import { useNavigate } from 'react-router-dom';

export function DeleteStationButton({ id }) {
  const navigate = useNavigate();

  function handleConfirmDelete() {
    // 1. Logique de suppression...
    // 2. Redirection programmatique vers l'accueil
    navigate('/');
  }

  return <button onClick={handleConfirmDelete}>Confirmer la suppression</button>;
}
```

2. **Lors du rendu conditionnel (`<Navigate>`) :** Utile pour rediriger une ancienne URL ou protéger une route si une condition n'est pas remplie. L'attribut `replace` évite de polluer l'historique du bouton "Précédent" du navigateur.

```
JavaScript
import { Navigate } from 'react-router-dom';

// Redirige immédiatement "/home" vers "/"
<Route path="/home" element={<Navigate to="/" replace />} />
```

6. Synthèse

1. **Routage SPA :** `react-router-dom` synchronise les vues React avec l'URL du navigateur sans jamais recharger le document HTML ni perdre l'état Redux.

2. **Composants Clés :** `<BrowserRouter>` initialise le routeur, `<Routes>` et `<Route>` associent chaque chemin (`path`) à un composant (`element`).
3. **Liens Sans Rechargement :** `<Link>` remplace `<a>`, et `<NavLink>` gère automatiquement le style de l'onglet actif.
4. **Hooks de Routage :** `useParams` lit les variables dynamiques de l'URL (`:id`), et `useNavigate` permet de changer de page par le code JavaScript.